SISTEMA INTEGRADO DE ATENDIMENTO E EXECUÇÃO DE SERVIÇOS

Tech Challenge — Fase 4

Arquitetura de microsserviços e gestão transacional distribuída

Danilo Evangelista Canato — RM 372411

Guilherme Fumagali Marques — RM 372421

PosTech FIAP — Arquitetura de Software

Novembro de 2026

Referências de Entrega

Participantes

Nome	RM	Discord	GitHub
Danilo Evangelista Canato	372411	@canatodanilo	kanatu
Guilherme Fumagali Marques	372421	@gfumafali	Guilherme-Fumagali

Repositórios

Cada microsserviço tem repositório próprio com código, Dockerfile, manifestos Kubernetes, Terraform do próprio banco e das próprias filas, pipelines, evidência de cobertura e documentação. A aplicação das fases anteriores permanece no repositório tech-challenge-1, que passou a ser o OS Service.

Repositório	Papel na Fase 4	Link
tech-challenge-1	OS Service (orquestrador da saga)	github.com/Guilherme-Fumagali/tech-challenge-1
oficina-billing-service	Billing Service (orçamento e pagamento)	github.com/Guilherme-Fumagali/oficina-billing-service
oficina-execution-service	Execution Service (execução e produção)	github.com/Guilherme-Fumagali/oficina-execution-service
oficina-auth-lambda	Autenticação por CPF com segundo fator	github.com/Guilherme-Fumagali/oficina-auth-lambda
oficina-infra-k8s	Plataforma: rede, EKS, gateway, New Relic	github.com/Guilherme-Fumagali/oficina-infra-k8s
oficina-infra-db	Banco da Fase 3 (migrado para o OS Service)	github.com/Guilherme-Fumagali/oficina-infra-db

Vídeo, contratos e documentação

Artefato	Link
Vídeo de demonstração	a publicar
OpenAPI — OS Service	https://redocly.github.io/redoc/?url=https://raw.githubusercontent.com/Guilherme-Fumagali/tech-challenge-1/develop/docs/openapi.json
OpenAPI — Billing Service	https://redocly.github.io/redoc/?url=https://raw.githubusercontent.com/Guilherme-Fumagali/oficina-billing-service/develop/docs/openapi.json
OpenAPI — Execution Service	https://redocly.github.io/redoc/?url=https://raw.githubusercontent.com/Guilherme-Fumagali/oficina-execution-service/develop/docs/openapi.json
AsyncAPI — mensagens da saga	https://github.com/Guilherme-Fumagali/tech-challenge-1/blob/develop/docs/asyncapi.yaml
Requisições do fluxo completo	https://github.com/Guilherme-Fumagali/tech-challenge-1/blob/develop/docs/demo.http
ADRs, diagramas e débitos técnicos	https://github.com/Guilherme-Fumagali/tech-challenge-1/tree/develop/docs/tech-challenge-4

1. Introdução e Contexto do Problema

A oficina passou a operar em escala nacional, com várias filiais. O volume de ordens de serviço e a interação entre atendimento, diagnóstico, execução e cobrança aumentaram, e com eles a necessidade de tolerar falhas parciais sem perder o andamento de uma ordem.

O objetivo da fase é refatorar a aplicação para uma arquitetura de microsserviços, com gestão transacional distribuída pelo padrão Saga e automação de build, testes e deploy por serviço. O enunciado exige no mínimo três microsserviços, cada um com repositório, infraestrutura e banco de dados próprios, ao menos um banco relacional e um não relacional, comunicação síncrona e assíncrona, integração com o Mercado Pago e a observabilidade construída na Fase 3.

1.1 Situação ao final da Fase 3

A aplicação era um monólito em Clean Architecture (Java 21, Spring Boot 3.4) com um banco PostgreSQL em RDS, implantado em EKS atrás de um API Gateway com autenticação por CPF em AWS Lambda e observabilidade no New Relic. O orçamento não era uma entidade: era calculado a partir dos itens da ordem de serviço, e a aprovação usava um token enviado por e-mail. Não havia pagamento, mensageria nem transações explícitas: cada gravação fazia commit isoladamente, de modo que a reserva de uma peça gravava o estoque e a ordem de serviço em commits separados.

1.2 Pontos das avaliações anteriores

As avaliações das fases anteriores registraram pontos de melhoria que foram tratados nesta fase:

Fase	Ponto registrado	Tratamento na Fase 4
1	Domain Storytelling não localizado na documentação	Domain Storytelling do fluxo distribuído na seção 2.4 e no README do OS Service
1	Execução do contêiner como root	Imagens com usuário não-root e securityContext restritivo nos três serviços
3	Autenticação somente por CPF, sem segundo fator	Código de uso único por e-mail após o CPF (seção 3.4)
3	Produção preparada e não provisionada	Mantida configurada e não provisionada por custo; registrada como débito (seção 13)

2. Divisão em Microsserviços

2.1 Critério de decomposição

A divisão seguiu a decomposição por subdomínio apresentada na disciplina de Estrutura de Microsserviços II, a partir dos contextos já identificados no Event Storming da Fase 1: atendimento (cliente, veículo e ordem de serviço), orçamento e aprovação, e execução. O enunciado sugere a mesma separação. A disciplina alerta que uma decomposição errada produz um monólito distribuído; por isso, dois acoplamentos do monólito foram avaliados antes de definir as fronteiras.

- **Estoque:** a peça é reservada no momento em que o mecânico a adiciona à ordem de serviço. Separar estoque e ordem de serviço em serviços diferentes transformaria a reserva e o estorno em passos distribuídos. Catálogo e estoque permaneceram no OS Service.
- **Orçamento:** o orçamento era um cálculo sobre os itens da ordem de serviço. No Billing Service ele passa a ser uma entidade própria, criada a partir de uma cópia (snapshot) dos itens enviada pelo orquestrador, o que preserva o conceito de snapshot de preço da Fase 1.

A migração seguiu o padrão Strangler Fig: o repositório do monólito foi mantido e perdeu, uma a uma, as responsabilidades extraídas para os novos serviços. O histórico das três fases e os testes do domínio foram preservados.

2.2 Responsabilidades

Serviço	Responsabilidades	Banco
OS Service (tech-challenge-1)	Cliente, veículo, funcionário, catálogo, estoque; abertura da OS, atualização de status, consulta de status e histórico; orquestração da saga; acompanhamento consolidado	PostgreSQL (RDS)
Billing Service	Geração do orçamento e envio por e-mail para aprovação; decisão do cliente; cobrança, registro e verificação do pagamento no Mercado Pago; estorno	PostgreSQL (RDS próprio)
Execution Service	Fila de execução; início e conclusão de diagnóstico e reparo; apontamentos do mecânico; registro de falha; comunicação de finalização	MongoDB (Amazon DocumentDB)

A atualização do status da ordem de serviço após o pagamento, atribuída pelo enunciado ao serviço de cobrança, ocorre por evento: o Billing Service publica PagamentoAprovado e o orquestrador, no OS Service, avança a ordem para EM_EXECUCAO. Assim, cada serviço altera apenas os próprios dados.

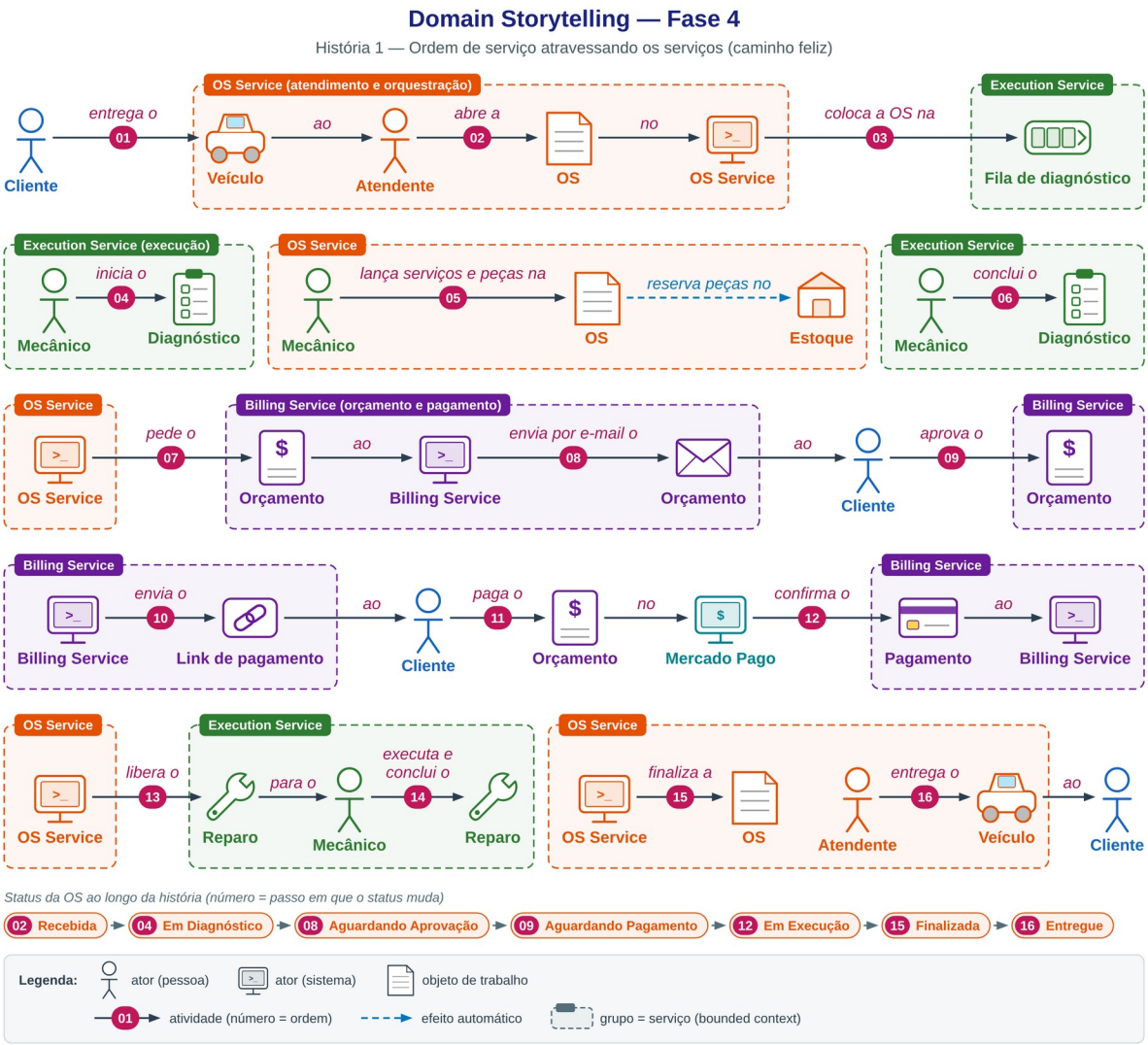
2.3 Código transversal

Os três serviços repetem o mesmo código de infraestrutura: validação do JWT, tratamento de erros, outbox, handler idempotente, trace e métricas. Esse conjunto corresponde ao microservice chassis visto na disciplina, copiado em cada repositório. Uma biblioteca compartilhada evitaria a cópia, mas criaria dependência de versão entre os serviços; a duplicação ficou registrada como débito técnico.

2.4 Domain Storytelling

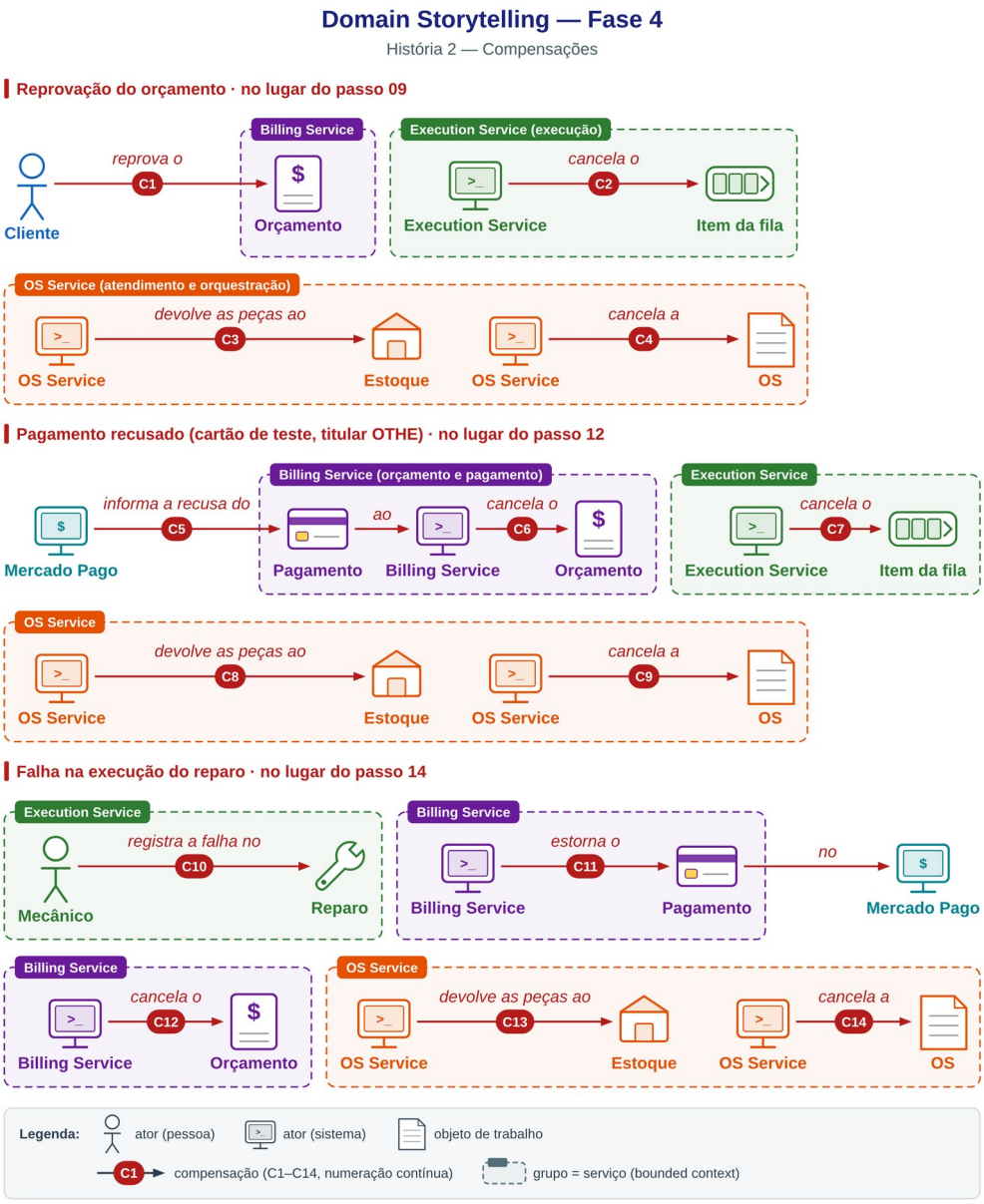
As figuras a seguir narram o fluxo de uma ordem de serviço atravessando os três serviços, com os atores, os objetos de trabalho e as atividades numeradas, e as histórias de compensação.

Figura 1 — Domain Storytelling — ordem de serviço atravessando os serviços



Fonte: elaborado pelos autores.

Figura 2 — Domain Storytelling — compensações



Fonte: elaborado pelos autores.

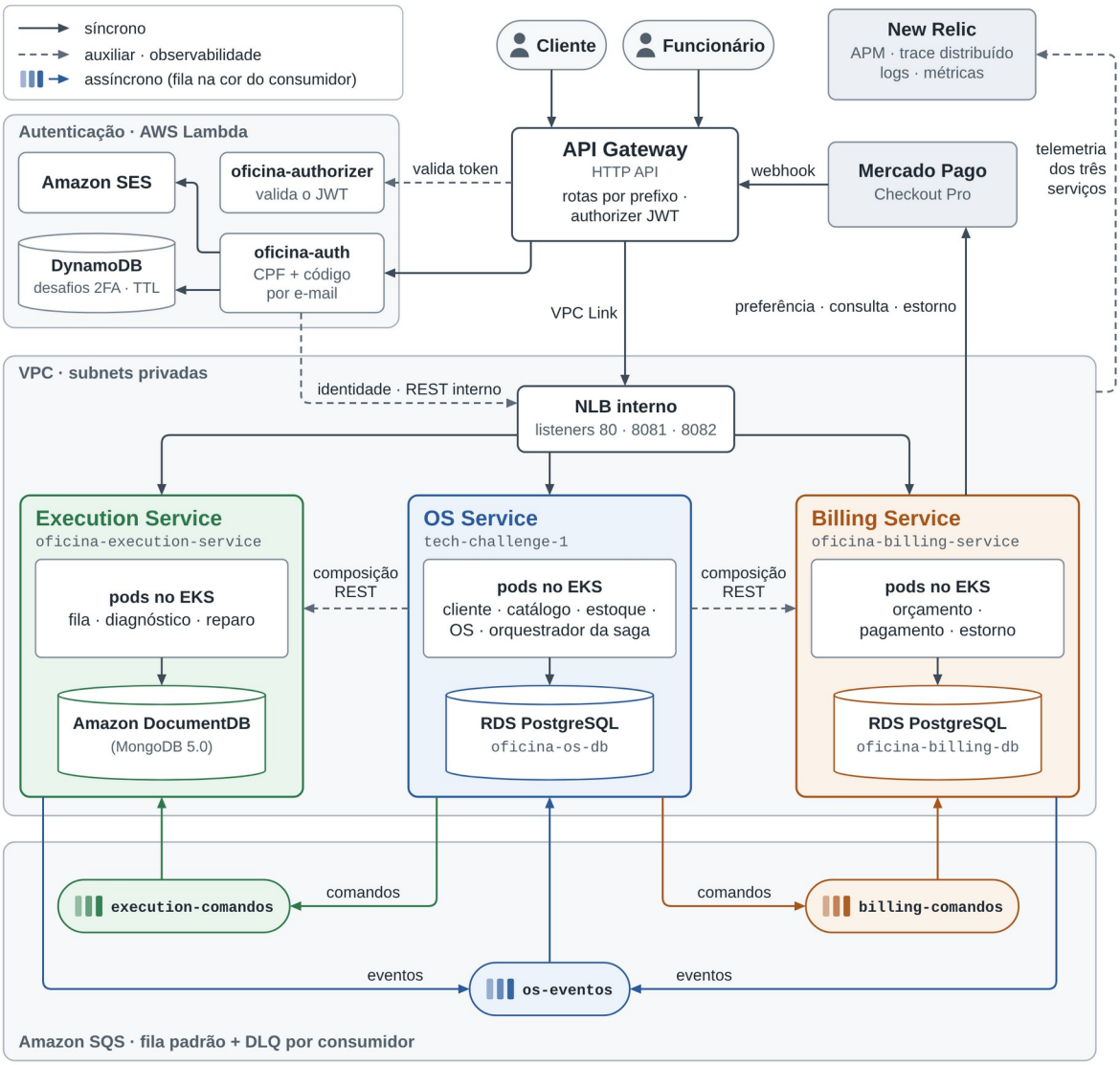
3. Arquitetura Final

3.1 Visão geral

A figura a seguir mostra os microsserviços, o banco de cada um e as formas de comunicação. O API Gateway é a única entrada externa. As requisições autenticadas seguem por VPC Link até um NLB interno, que tem um listener por serviço. Os serviços rodam em pods no EKS, em subnets privadas, e cada um acessa apenas o próprio banco. A coordenação

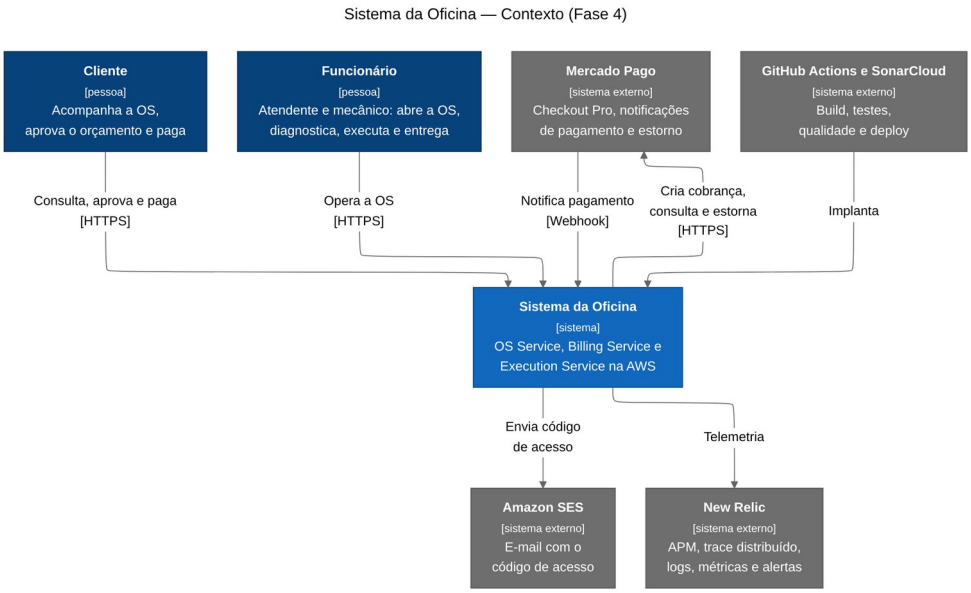
da saga usa filas SQS: os comandos do orquestrador vão para a fila de cada participante, e as respostas e eventos voltam por uma fila do OS Service.

Figura 3 — Arquitetura geral: microsserviços, bancos e comunicação



Fonte: elaborado pelos autores.

Figura 4 — Diagrama de contexto (C4, nível 1)



Fonte: elaborado pelos autores.

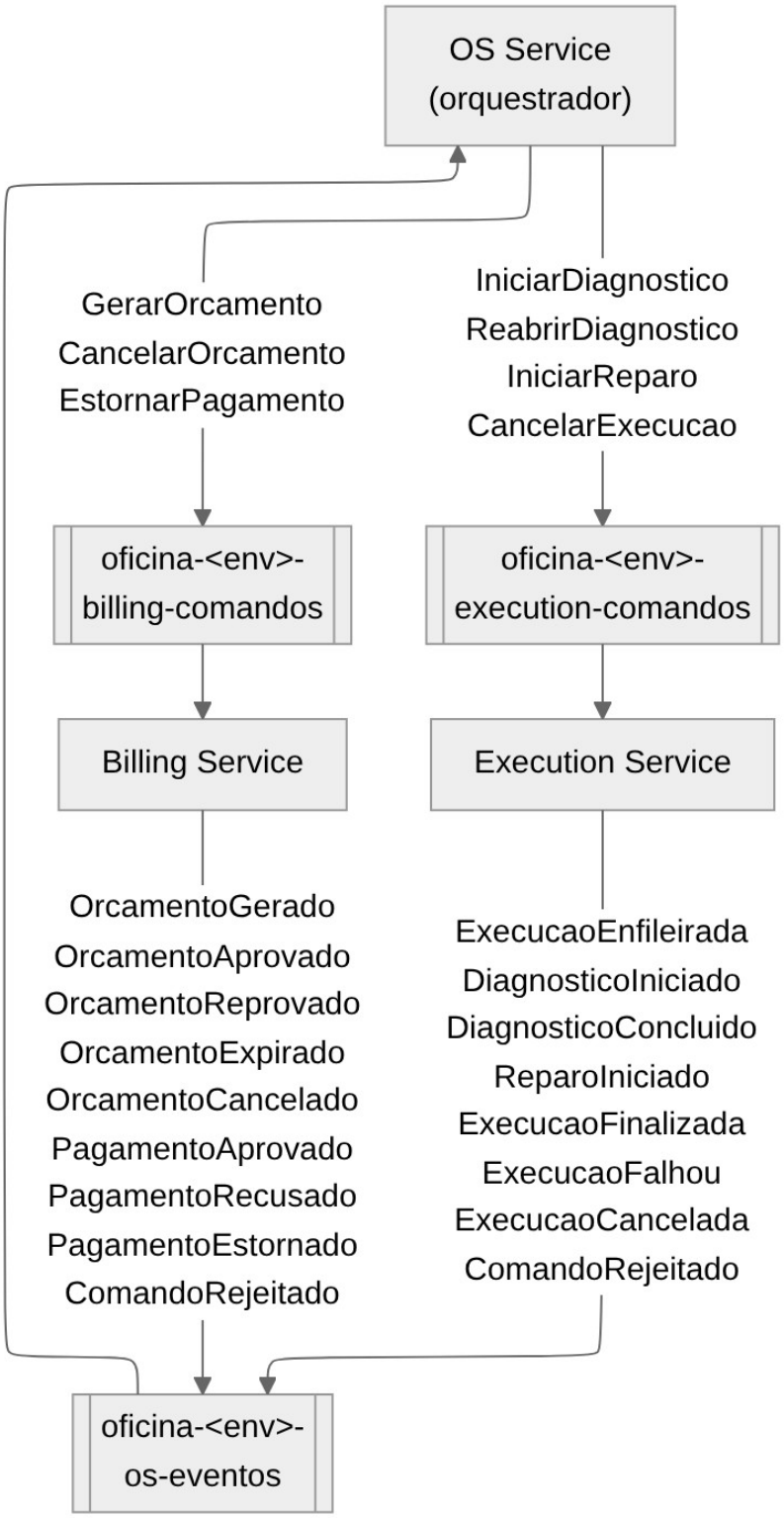
3.2 Comunicação síncrona e assíncrona

A disciplina de Estrutura de Microsserviços distingue a comunicação síncrona (request/response) da assíncrona por mensageria, com canais ponto a ponto ou publish-subscribe. A comunicação síncrona foi usada onde a resposta precisa ser imediata e o dado precisa ser atual; a assíncrona, nos passos da saga.

Interação	Estilo	Justificativa
Cliente e funcionário → serviços	REST síncrono via API Gateway	interação humana com resposta imediata
OS → Billing e Execution (acompanhamento)	REST síncrono, composição	visão consolidada sob demanda (seção 3.3)
Lambda de autenticação → OS	REST síncrono interno	a autenticação precisa do status atual da identidade
Orquestrador → participantes	SQS, comandos ponto a ponto	desacopla disponibilidade; mensagens aguardam o serviço voltar
Participantes → orquestrador	SQS, eventos para a fila do OS	eventos de negócio e respostas aos comandos
Mercado Pago → Billing	Webhook assinado + consulta	notificação do pagamento com verificação na API do gateway

As mensagens usam um envelope comum com identificador da mensagem, tipo, versão, identificador da saga, identificador da ordem de serviço, identificador do comando que a originou, origem e data. O contrato completo está publicado no AsyncAPI de cada serviço. A figura a seguir resume as filas e os tipos de mensagem.

Figura 5 — Filas e mensagens da saga



Fonte: elaborado pelos autores.

3.3 Composição de API no acompanhamento

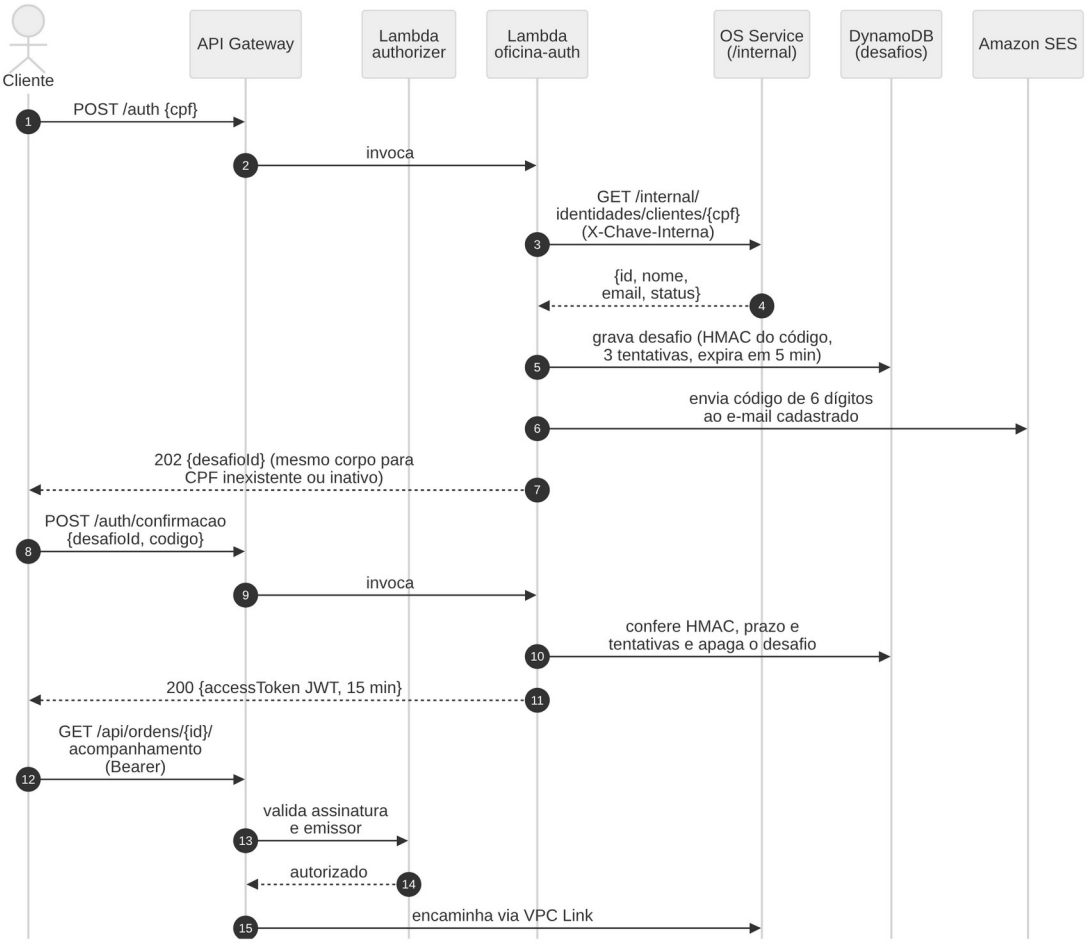
O endpoint GET /api/ordens/{id}/acompanhamento do OS Service consolida o status e o histórico da ordem de serviço (dados locais) com o orçamento e o pagamento (Billing) e a etapa da execução (Execution), consultados em paralelo. Segue o padrão API Composition da aula 5 de Estrutura de Microsserviços. Cada chamada tem tempo limite de 2 segundos e circuit breaker (Resilience4j). Se um serviço não responde, a resposta vem parcial, indicando o que ficou de fora, em vez de erro.

3.4 Borda e autenticação com segundo fator

O API Gateway encaminha cada prefixo de caminho ao serviço dono do recurso: /api/orcamentos e /api/pagamentos ao Billing Service, /api/execucoes ao Execution Service e os demais caminhos /api ao OS Service. As rotas públicas são declaradas uma a uma: consulta de status da ordem de serviço, decisão do orçamento por token enviado por e-mail, webhook do Mercado Pago e health check. As demais passam pelo Lambda authorizer, que valida o JWT.

A autenticação passou a ter dois passos. O cliente informa o CPF e recebe um código de seis dígitos no e-mail cadastrado; em seguida informa o código e recebe o JWT de 15 minutos. O código não fica salvo em texto: a tabela DynamoDB guarda apenas um hash, que expira em cinco minutos e aceita no máximo três tentativas. A resposta do primeiro passo é a mesma para CPF cadastrado, inexistente ou inativo, e qualquer falha no segundo passo devolve o mesmo 401. O Lambda deixou de consultar o banco da aplicação: a identidade vem de um endpoint interno do OS Service, alcançado só de dentro da VPC e protegido por chave compartilhada.

Figura 6 — Autenticação com segundo fator



Fonte: elaborado pelos autores.

4. Saga Pattern

4.1 Estratégia escolhida: orquestração

As aulas de SAGA apresentam duas estratégias. Na coreografia, cada serviço reage aos eventos dos outros; a implementação é simples e o acoplamento é fraco, mas não existe um lugar que descreva a saga inteira, há risco de dependências cíclicas e cada participante precisa assinar os eventos que o afetam. Na orquestração, um componente envia comandos aos participantes; as dependências passam a ter mão única e a lógica fica centralizada, ao custo de concentrar complexidade e de criar um ponto do qual o fluxo depende. A aula 2 registra a recomendação de Chris Richardson: coreografia para sagas simples e orquestração nos demais casos.

A saga da oficina tem sete etapas, duas esperas por pessoas (aprovação e pagamento), atividades do mecânico que podem durar dias e três caminhos de falha, cada um com

compensações em ordem diferente. Por isso foi adotada a orquestração, com o orquestrador no OS Service, dono da ordem de serviço, como no projeto da aula 5, em que a saga fica no serviço de pedidos.

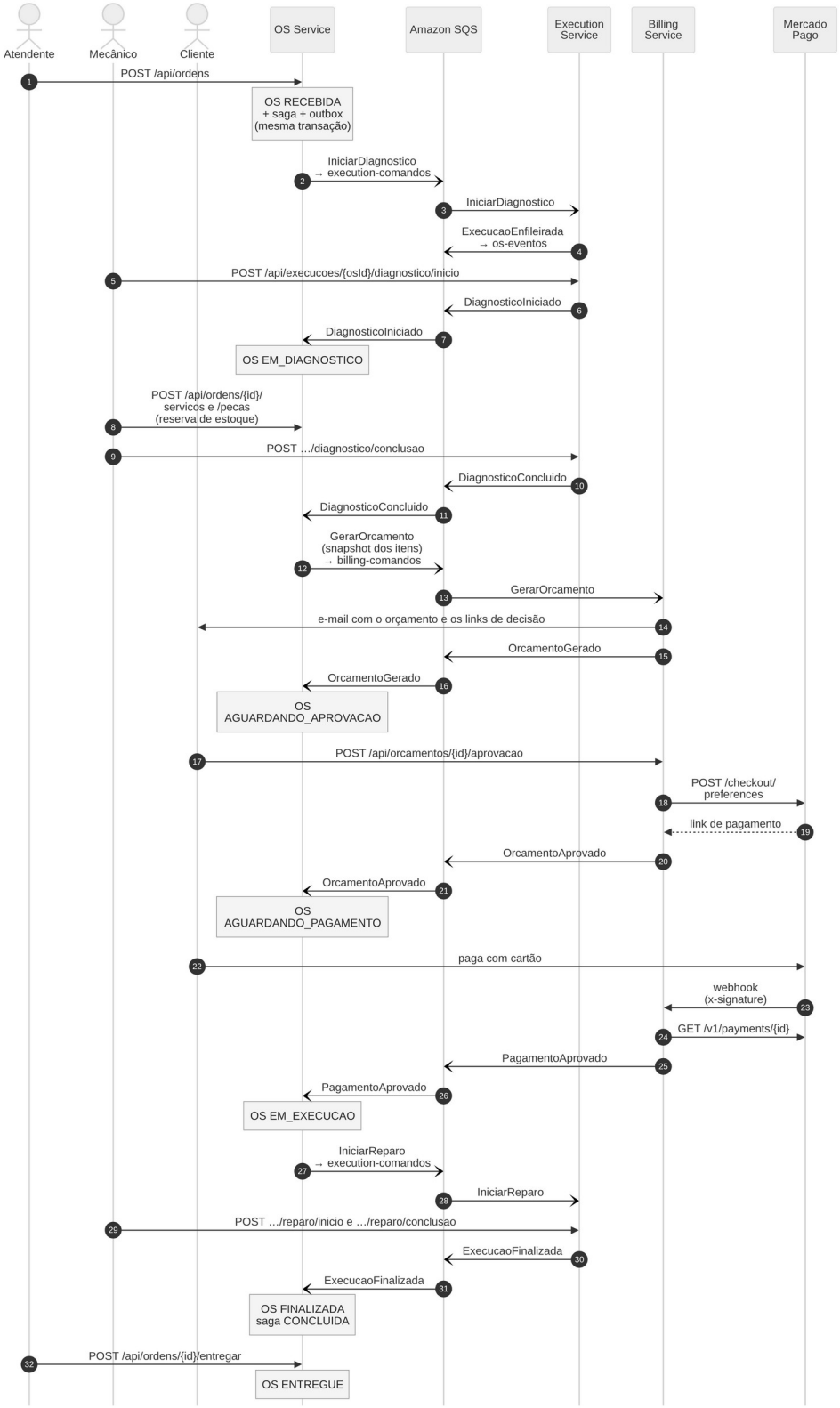
Critério	Coreografia	Orquestração (adotada)
Onde está a definição do fluxo	distribuída entre os serviços	classe SagaOrdemServico e tabela de transições
Conhecimento entre serviços	cada serviço assina eventos dos outros	Billing e Execution só respondem a comandos
Compensações em ordens diferentes	cada serviço precisa conhecer a ordem	definidas por motivo no orquestrador
Consulta do andamento	reconstruída a partir de eventos	estado persistido e endpoint /api/ordens/{id}/saga
Ponto de atenção	dependências cíclicas	indisponibilidade do OS pausa o avanço; mensagens aguardam na fila

O AWS Step Functions, citado na aula 2, também foi considerado e descartado: as esperas pela decisão do cliente e pelo pagamento exigiriam configuração adicional, e a lógica da saga sairia do código coberto pelos testes do pipeline.

4.2 Fluxo

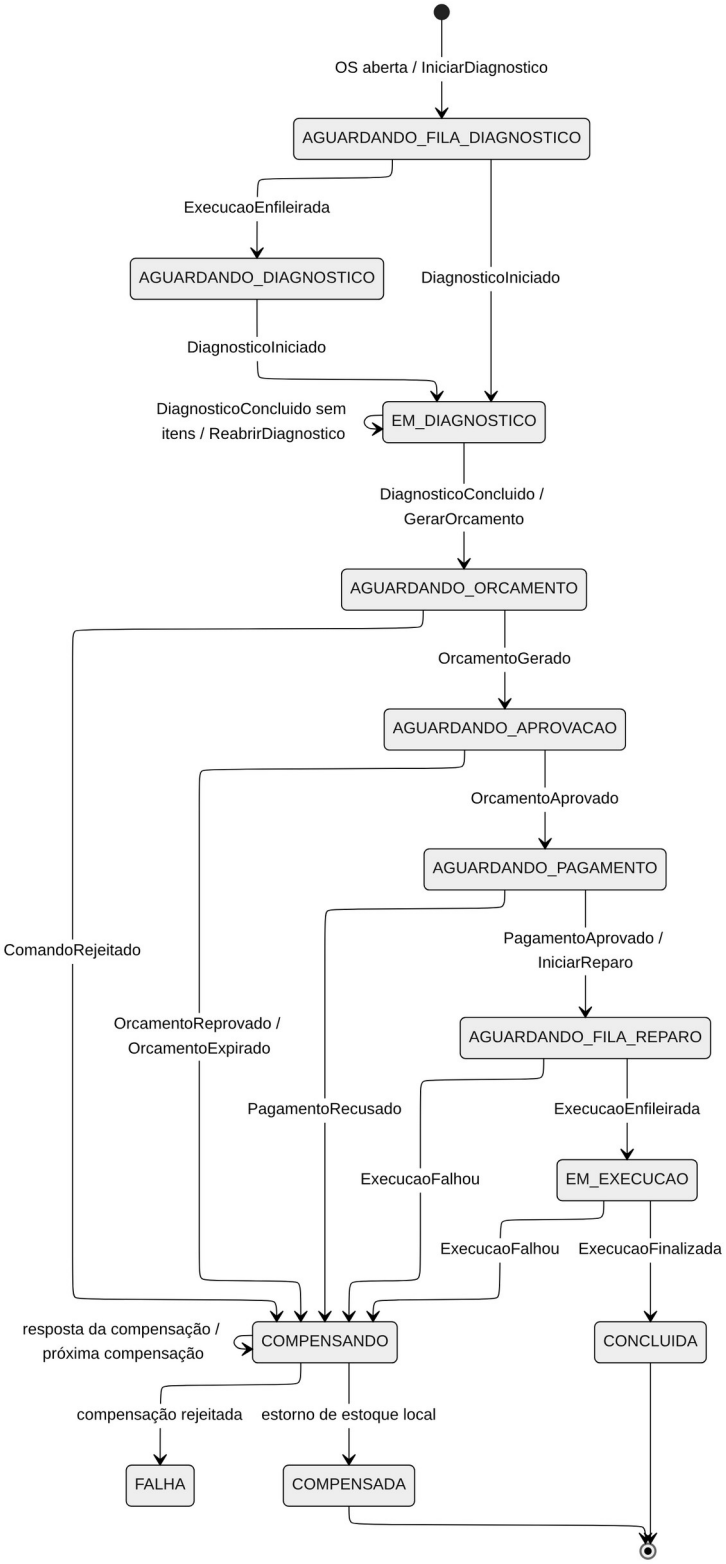
O estado da saga é persistido no banco do OS Service. A lógica de transição é uma classe de domínio sem dependência de framework, que recebe um evento e devolve a decisão: transição da ordem de serviço, comandos a enviar e compensação local. Cada linha da tabela de transições tem um teste unitário. As figuras a seguir mostram o fluxo completo, os estados da saga e os status da ordem de serviço.

Figura 7 — Saga da ordem de serviço — fluxo completo



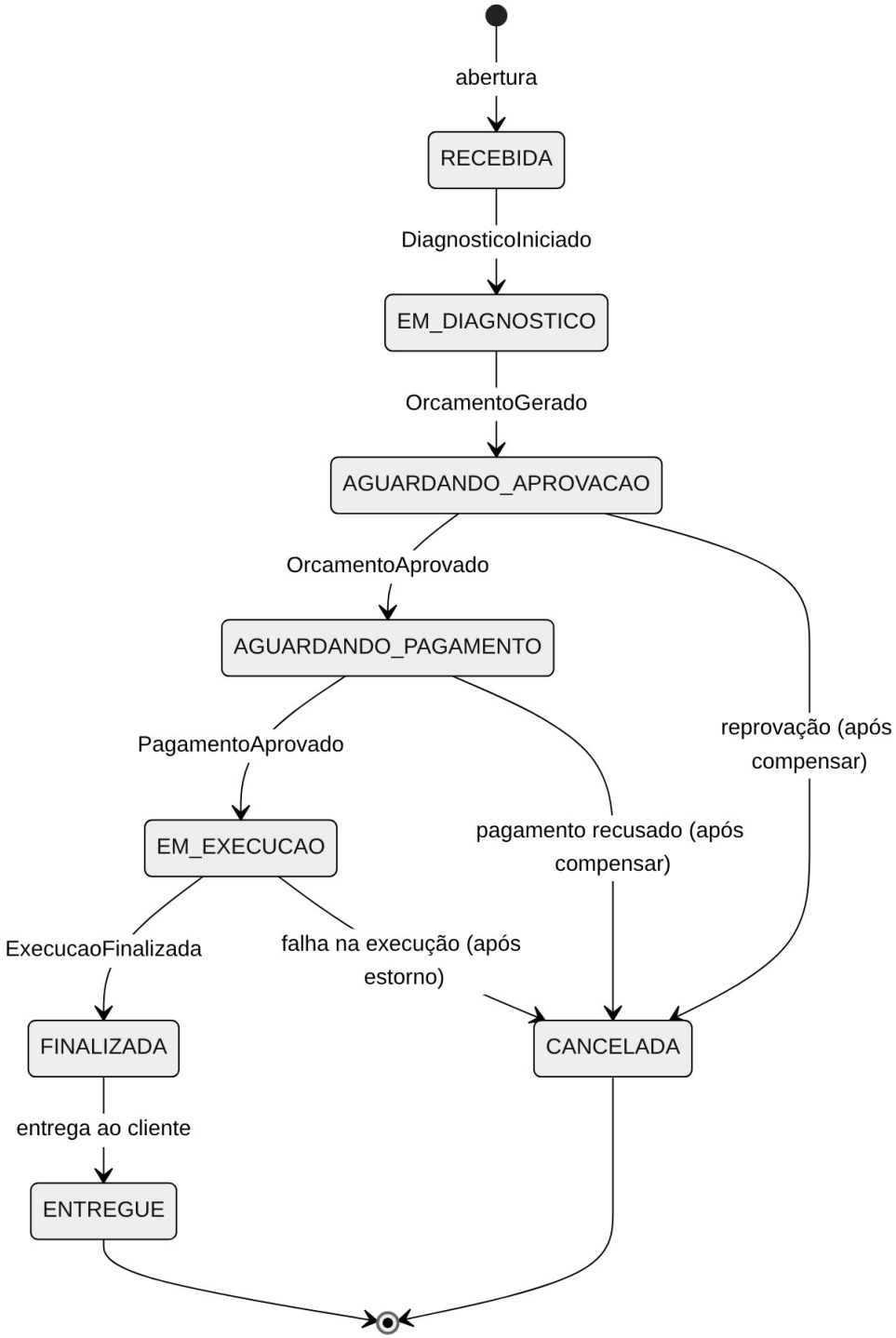
Fonte: elaborado pelos autores.

Figura 8 — Máquina de estados da saga



Fonte: elaborado pelos autores.

Figura 9 — Status da ordem de serviço na Fase 4



Fonte: elaborado pelos autores.

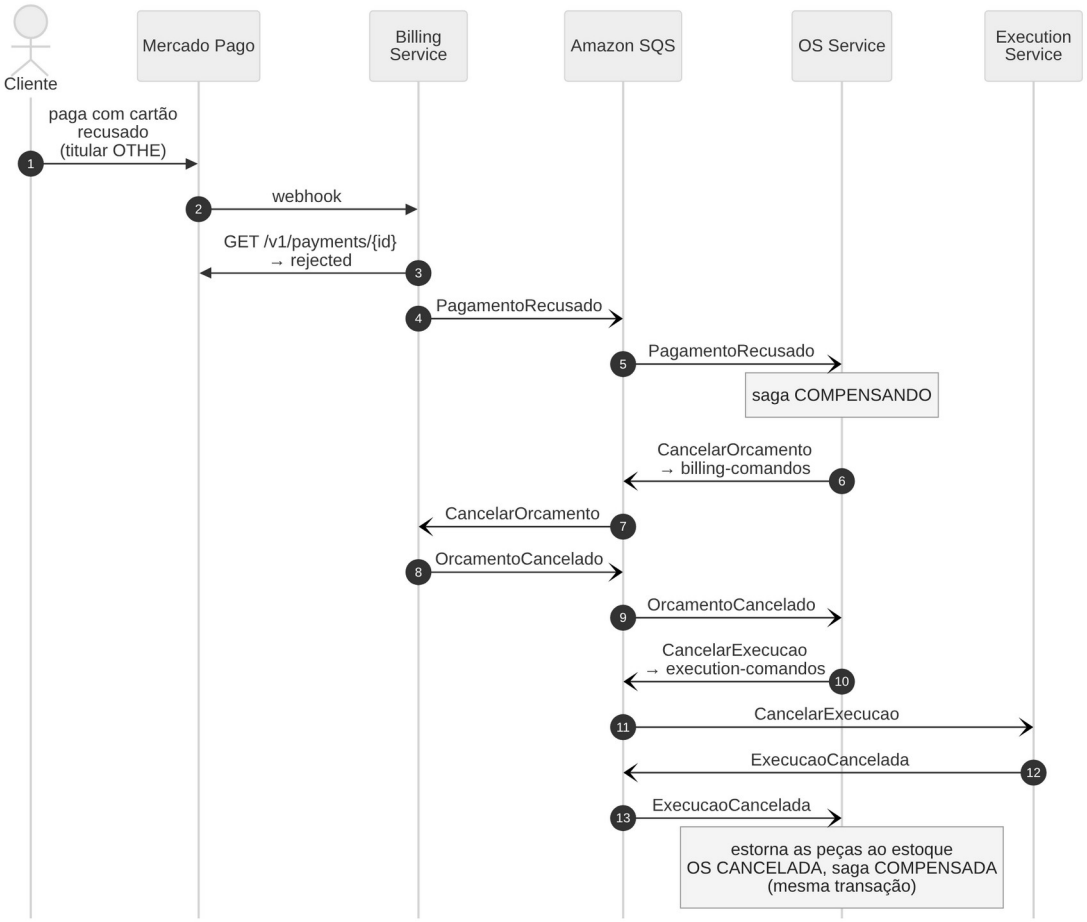
4.3 Compensações

Em caso de falha, o orquestrador executa as compensações dos passos já realizados. A ordem segue a contramedida pessimistic view da aula 3 de SAGA: primeiro o que impede trabalho novo (cancelar o orçamento, cancelar a execução) e por último o que devolve recurso

(estornar o pagamento, devolver as peças ao estoque). O estorno de estoque é uma transação local do OS Service, executada junto com o cancelamento da ordem de serviço.

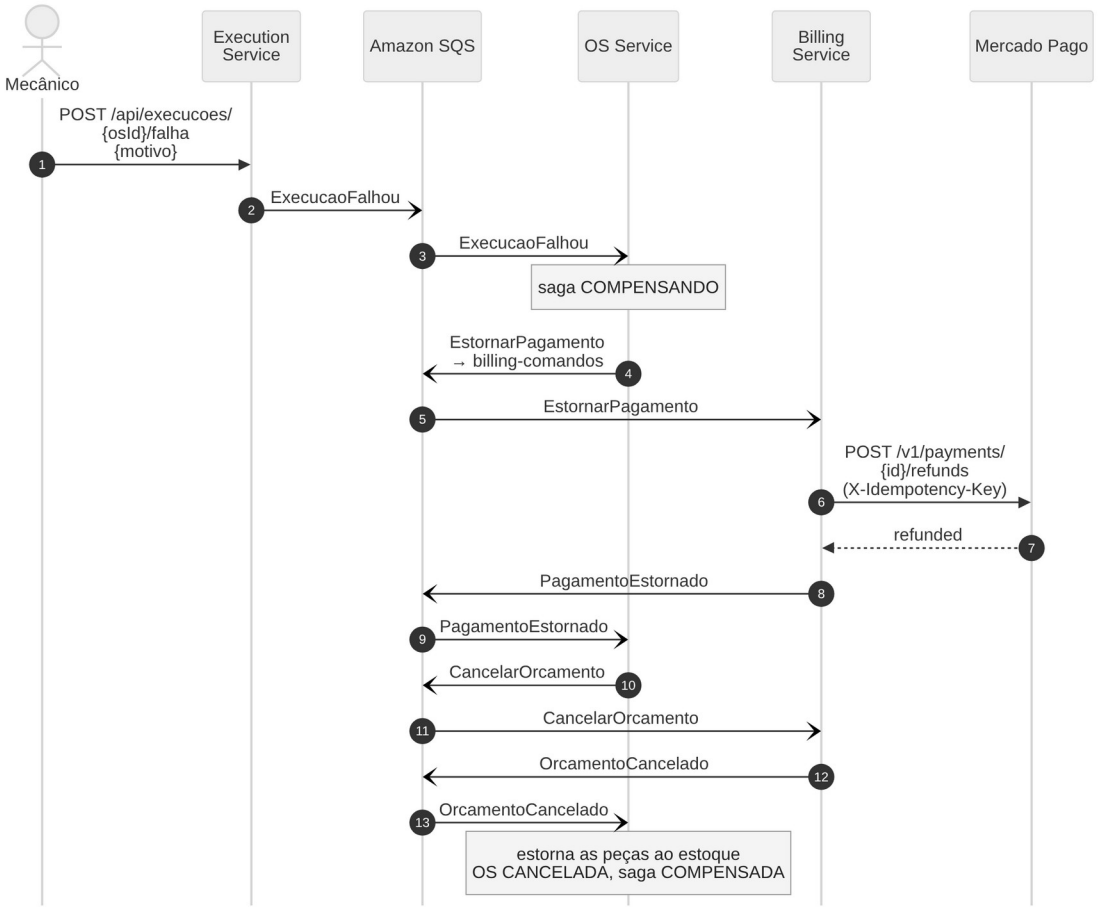
Falha	Compensações, em ordem	Resultado
Orçamento reprovado ou expirado	cancelar execução → estornar estoque	OS CANCELADA
Orçamento rejeitado pelo Billing	cancelar execução → estornar estoque	OS CANCELADA
Pagamento recusado ou expirado	cancelar orçamento → cancelar execução → estornar estoque	OS CANCELADA
Falha na execução	estornar pagamento no Mercado Pago → cancelar orçamento → estornar estoque	OS CANCELADA
Diagnóstico concluído sem itens	reabrir diagnóstico	volta a EM_DIAGNOSTICO
Participante indisponível	nenhuma: reenvio do comando após o prazo (até 3 vezes)	saga retoma ou vai para FALHA com alerta

Figura 10 — Compensação por pagamento recusado



Fonte: elaborado pelos autores.

Figura 11 — Compensação por falha na execução, com estorno



Fonte: elaborado pelos autores.

Falha técnica, como um serviço fora do ar, não gera compensação: a mensagem espera na fila e é processada quando o serviço volta. Só falhas de negócio (reprovação, recusa do pagamento, falha no reparo) desfazem a saga.

4.4 Falta de isolamento: contramedidas

Uma saga oferece atomicidade, consistência e durabilidade, mas não isolamento (aula 3 de SAGA). Duas contramedidas da aula foram aplicadas:

- **Reread value:** antes de aplicar cada evento, o orquestrador relê a ordem de serviço e a saga. Uma trava otimista (@Version) impede que duas atualizações simultâneas sobrescrevam uma à outra; quando isso acontece, a mensagem é processada de novo. A mesma trava protege o estoque, que no monólito podia perder atualizações.
- **Pessimistic view:** a ordem das compensações garante que nenhum trabalho novo comece sobre uma ordem de serviço em cancelamento antes de o recurso ser devolvido.

4.5 Garantias de entrega

Gravar no banco e publicar na fila são operações em sistemas diferentes. No projeto da aula 5, as duas ficam no mesmo @Transactional, mas a anotação só controla o banco: se a publicação falhar depois do commit, a mensagem se perde. Por isso foram usados o Transaction Outbox, citado na aula 1 de Estrutura de Microsserviços II, e o handler idempotente da aula 5 de Estrutura de Microsserviços:

- **Transação local:** cada caso de uso que grava dados roda em uma única transação. Isso também corrigiu a reserva de peças do monólito, que gravava o estoque e a ordem de serviço em momentos diferentes.
- **Transaction Outbox:** a mensagem é gravada em uma tabela outbox na mesma transação da alteração, e um job a envia para a fila logo em seguida.
- **Handler idempotente:** o consumidor registra o identificador de cada mensagem processada; se a mesma mensagem chegar de novo, ela é ignorada.
- **Prazos e DLQ:** se um participante não responde no prazo, o comando é reenviado, e o participante responde sem repetir o efeito. Uma mensagem que falha cinco vezes vai para uma fila de erro (DLQ), que gera alerta.

5. Banco de Dados por Serviço

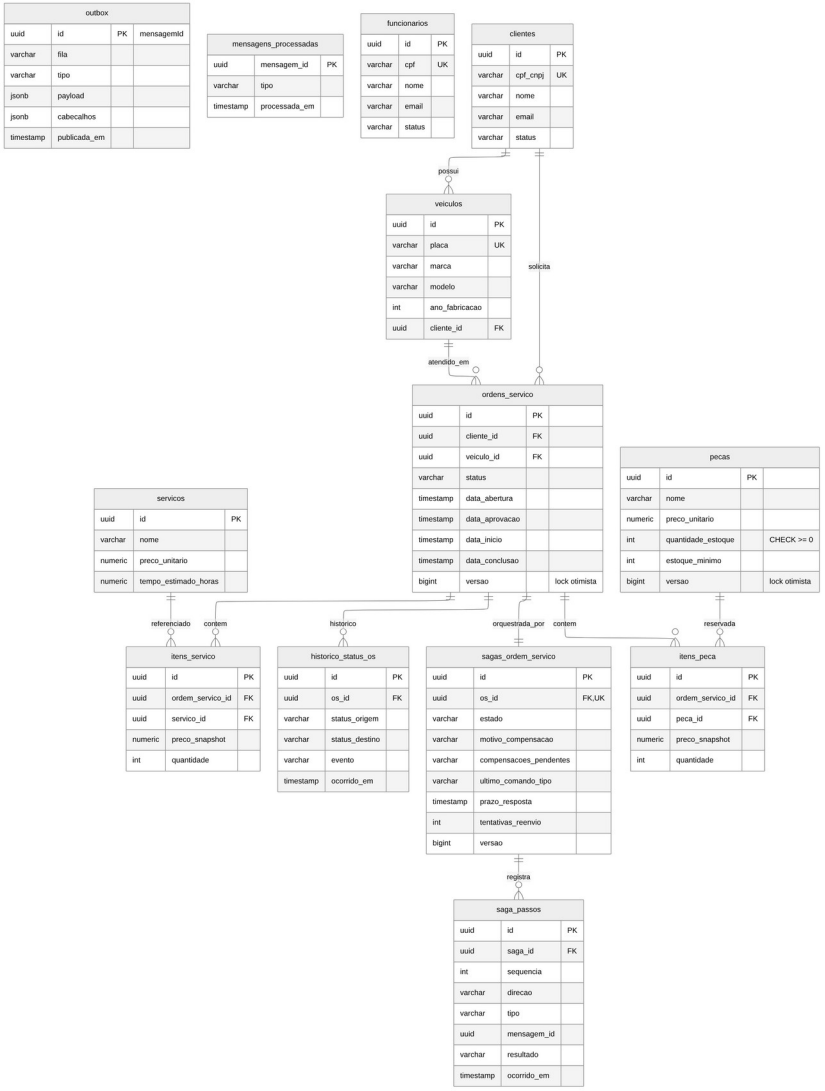
A aula 5 de Estrutura de Microsserviços classifica o banco compartilhado como anti-padrão e adota o banco por serviço, aceitando a necessidade de composição, CQRS e saga. A aula 7 de Data Engineering acrescenta que cada serviço pode usar um tipo de banco diferente. Cada serviço recebe apenas as próprias credenciais, e o Terraform de cada banco está no repositório do serviço dono.

Serviço	Banco	Por quê	Alternativas descartadas
OS Service	PostgreSQL 16 (RDS)	dados muito relacionados; regras garantidas pelo banco (estoque não negativo, CPF e placa únicos); reserva da peça e item da OS na mesma transação	NoSQL perderia essas regras e a transação entre tabelas
Billing Service	PostgreSQL 16 (RDS próprio)	dinheiro exige regras rígidas: um pagamento aprovado por orçamento, sem notificação processada duas vezes; histórico para conferência	MongoDB guardaria os dados, mas as regras ficariam só no código; usar o banco do OS juntaria falhas e credenciais
Execution Service	MongoDB 5.0 (Amazon DocumentDB)	cada execução é lida e gravada inteira; etapas, apontamentos e histórico variam e ficam dentro do	Redis: voltado a cache; Cassandra: complexo demais para o volume; Neo4j: relacionamentos não

		documento; serviço gerenciado na VPC	são o foco; MongoDB fora da AWS ou dentro do cluster: mais operação
Autenticação	DynamoDB	código de login buscado pela chave e válido por 5 minutos; o próprio banco apaga o registro vencido (TTL)	Redis (ElastiCache): servidor pago ligado o tempo todo para um dado de minutos

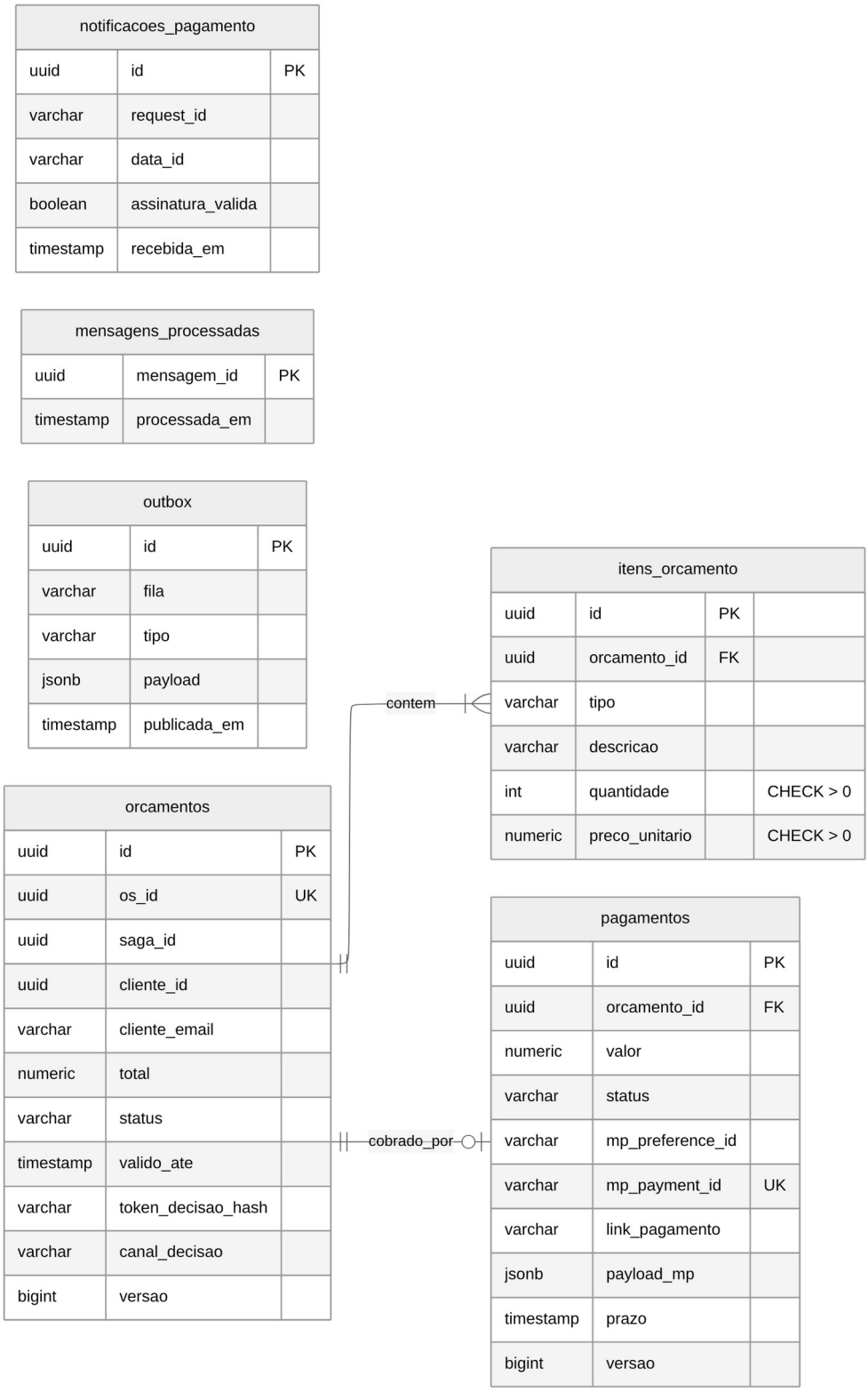
A escolha do MongoDB para o Execution Service segue a aula 2 de Data Engineering, que associa o modelo de documentos a dados semiestruturados cuja estrutura varia ao longo do tempo e recomenda incorporar os relacionamentos no próprio documento. O DocumentDB é compatível com a API do MongoDB, mas não idêntico; por isso o código usa apenas recursos suportados pela versão 5.0, e a conexão usa TLS.

Figura 12 — Modelo relacional do OS Service



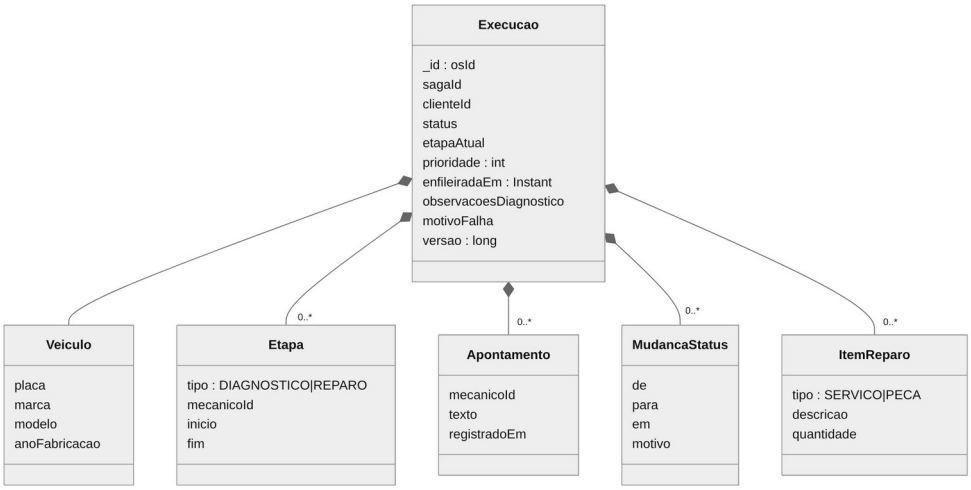
Fonte: elaborado pelos autores.

Figura 13 — Modelo relacional do Billing Service



Fonte: elaborado pelos autores.

Figura 14 — Documento de execução no MongoDB



Fonte: elaborado pelos autores.

6. Integração com o Mercado Pago

O Billing Service usa o Checkout Pro do Mercado Pago pela API REST. Quando o cliente aprova o orçamento, o serviço cria uma cobrança e envia ao cliente o link de pagamento. O endereço para notificações vai em cada cobrança, porque a URL do API Gateway muda sempre que o ambiente é criado.

Operação	Chamada	Proteções
Criar cobrança	POST /checkout/preferences	chave de idempotência; nova tentativa e circuit breaker
Receber notificação	POST /api/pagamentos/webhooks/mercado-pago	assinatura do Mercado Pago validada; status confirmado por consulta
Consultar pagamento	GET /v1/payments/{id}	token do vendedor
Reconciliar	GET /v1/payments/search	job a cada 60 s para pagamentos pendentes
Estornar	POST /v1/payments/{id}/refunds	compensação quando a execução falha depois do pagamento

A reconciliação por consulta faz parte do fluxo: segundo a documentação do Mercado Pago, pagamentos com credenciais de teste não disparam notificações, e uma notificação pode se perder se o serviço estiver fora do ar. Nos testes automatizados e na execução local, um gateway simulado substitui o Mercado Pago e permite executar a saga completa. Na demonstração, o cartão de teste com titular APRO aprova o pagamento e o titular OTHE o recusa, acionando a compensação.

7. Tecnologias Utilizadas

Tecnologia	Uso	Justificativa
Java 21 e Spring Boot 3.4	três serviços	continuidade das fases anteriores; mesma linguagem do projeto de referência das aulas
Clean Architecture / hexagonal	três serviços	domínio sem framework; portas para banco, fila e gateway
Amazon SQS (Spring Cloud AWS 3.3)	mensageria da saga	gerenciado, DLQ nativa, acesso por IAM; usado na aula 5 de Estrutura de Microsserviços
PostgreSQL 16 (RDS)	OS e Billing	seção 5
MongoDB / Amazon DocumentDB	Execution	seção 5
DynamoDB	segundo fator	seção 5
Resilience4j	composição e Mercado Pago	tempo limite, nova tentativa e circuit breaker
AWS Lambda + API Gateway (HTTP API)	borda e autenticação	mantidos da Fase 3
Amazon EKS + Kustomize	execução e deploy dos serviços	manifestos por serviço com overlays de homologação e produção
EKS Pod Identity	acesso dos pods às filas	role por serviço sem chave estática
Terraform	plataforma e infraestrutura de cada serviço	infraestrutura como código
GitHub Actions + SonarCloud	CI/CD e qualidade	pipeline independente por serviço com quality gate
JUnit 5, Mockito, Testcontainers, LocalStack, WireMock, Cucumber	testes	unitários, integração com dependências reais em contêiner, BDD
New Relic	observabilidade	APM, trace distribuído, logs e alertas da Fase 3
Mercado Pago Checkout Pro	pagamento	exigência do enunciado

8. Testes e Qualidade

Todos os serviços têm testes unitários do domínio e dos casos de uso, testes de integração com Testcontainers (PostgreSQL ou MongoDB e LocalStack para a SQS) e teste de contrato da API, que compara o OpenAPI gerado com o versionado. A cobertura mínima de 80% é verificada pelo JaCoCo e interrompe o build; o SonarCloud aplica o quality gate em cada pull request.

Serviço	Unitários	Integração	BDD	Total	Linhas	Ramos
OS Service	282	44	7	333	95,3%	85,1%
Billing Service	146	38	—	184	98,6%	89,1%
Execution Service	266	26	—	292	99,4%	95,2%
Lambda de autenticação	85	26	—	111	92,7%	90,4%

8.1 BDD

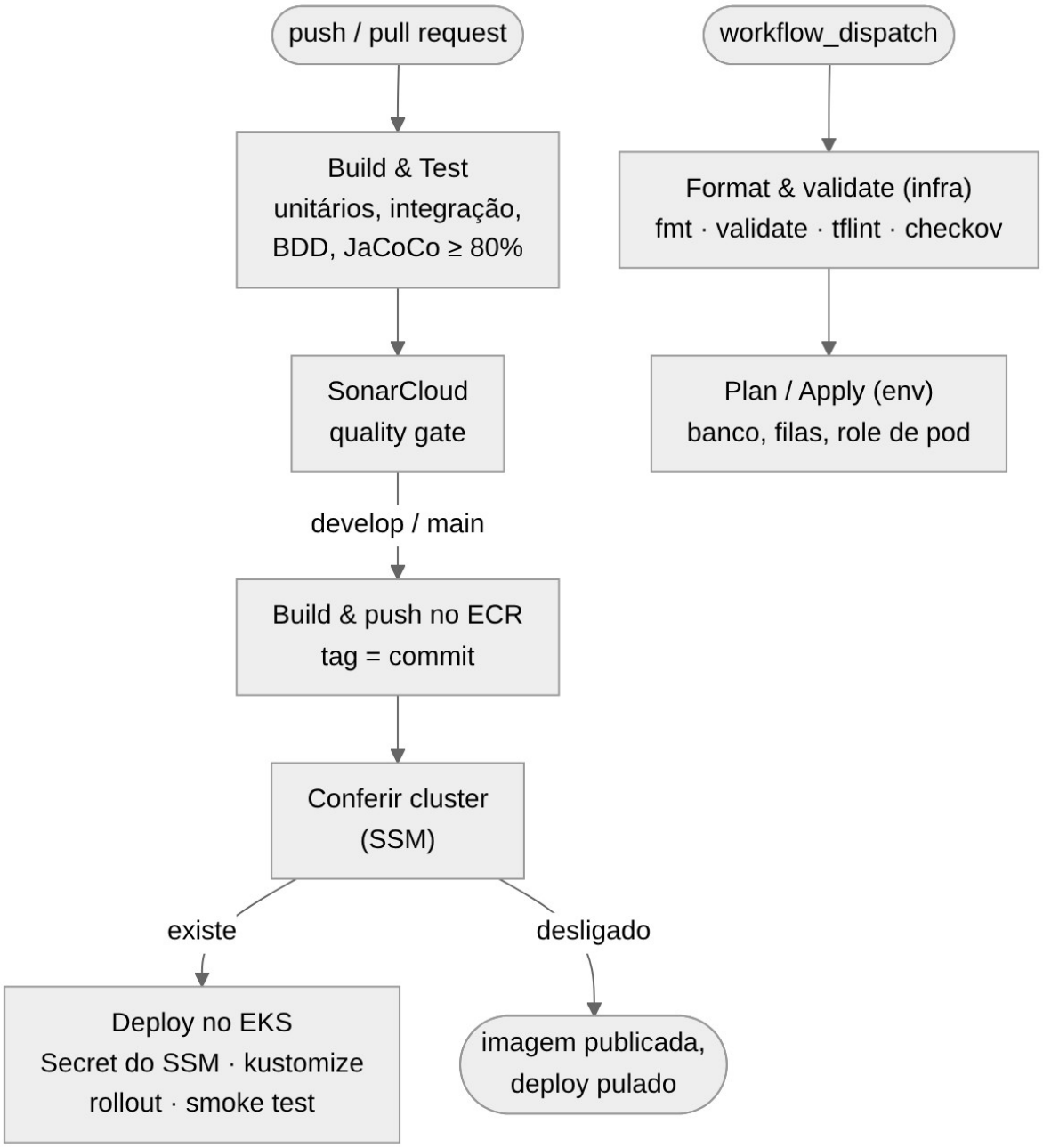
O fluxo completo da saga é especificado em Gherkin, em português, e executado pelo Cucumber no pipeline do OS Service. Os cenários usam a linguagem do negócio. Os participantes são simulados por passos que publicam eventos na fila do orquestrador, com PostgreSQL e SQS reais em contêiner. Os cenários cobrem o fluxo completo até a entrega, a reprovação do orçamento, o pagamento recusado, a falha na execução com estorno, o diagnóstico sem itens, a mensagem duplicada e o reenvio de comando com prazo vencido.

```
# language: pt
Funcionalidade: Saga da ordem de serviço
  Cenário: Pagamento recusado cancela a ordem e devolve as peças
    Dado uma ordem de serviço com orçamento aprovado e uma peça reservada
    Quando o Mercado Pago recusar o pagamento
    Então o orçamento é cancelado
    E a execução é cancelada
    E a peça volta ao estoque
    E a ordem de serviço fica CANCELADA
```

9. CI/CD e Implantação

Cada serviço tem pipeline próprio no GitHub Actions. Todo push, inclusive nas branches de pull request, executa o build com os testes e a verificação de cobertura; a análise do SonarCloud roda no pull request e na main. Em develop e main, a imagem é publicada no ECR do serviço e implantada no EKS com kubectl apply -k sobre o overlay do ambiente; o pipeline aguarda o rollout e executa um teste de fumaça no endpoint de readiness. A autenticação na AWS usa OIDC, sem chave estática, e os segredos são lidos do SSM no momento do deploy.

Figura 15 — Pipeline de cada serviço



Fonte: elaborado pelos autores.

As branches main e develop dos repositórios são protegidas: merge somente por pull request com uma aprovação e com os checks Build & Test, SonarCloud e Format & validate (infra) aprovados, sem force push nem exclusão e sem exceção para administradores. A produção exige revisor no environment do GitHub.

9.1 Infraestrutura por serviço

A plataforma compartilhada (rede, EKS, NLB, API Gateway, repositórios de imagem e integração do New Relic) permanece no repositório oficina-infra-k8s. O banco, as filas de entrada com DLQ e a role de pod de cada serviço ficam no diretório infra/ do próprio serviço, com estado Terraform separado. Os manifestos Kubernetes (Deployment, Service, HPA, PodDisruptionBudget, ServiceAccount e ConfigMap) também passaram para cada serviço, o que encerrou o débito da Fase 3 em que a infraestrutura reaplicava o Deployment da aplicação. Os pods acessam a SQS por EKS Pod Identity, com uma role por serviço limitada por permissions boundary.

Ordem	Passo	Repositório
1	Backend de estado (bucket e tabela de lock)	oficina-infra-k8s (bootstrap)
2	Repositórios de imagem e parâmetros compartilhados	oficina-infra-k8s (base)
3	Rede, EKS, NLB e API Gateway	oficina-infra-k8s (cluster)
4	Banco, filas e role de pod de cada serviço	repositório de cada serviço (infra/)
5	Funções de autenticação	oficina-auth-lambda
6	Rotas de autenticação no gateway	oficina-infra-k8s (cluster)
7	Deploy dos serviços	pipeline de cada serviço

10. Monitoramento e Rastreamento dos Fluxos Distribuídos

A observabilidade da Fase 3 foi estendida aos três serviços, cada um com sua aplicação no New Relic. O agente Java liga automaticamente as chamadas HTTP entre os serviços. Para que o rastreamento também atravesse a fila, o contexto do trace vai junto com a mensagem, e o consumidor continua o mesmo trace. Assim, um único trace mostra o caminho da abertura da ordem de serviço até os participantes. Logs e transações também levam os.id e saga.id, o que permite filtrar tudo o que aconteceu com uma ordem de serviço.

Sinal	Exemplos	Uso
Métricas da saga	oficina.saga.iniciadas, concluidas, compensadas{motivo}, falhas	dashboard Saga; alerta de falha e de pico de compensações
Mensageria	oficina.mensageria.publicadas, consumidas{resultado}, dlq.mensagens	alerta crítico com mensagem na DLQ
Pagamento	oficina.pagamento.resultados, oficina.integracao.mercadopago	dashboard Negócio; alerta de falha no Mercado Pago
Execução	oficina.execucao.fila.tamanho{etapa}	fila por etapa no dashboard Saga
APM e Kubernetes	latência p95, taxa de 5xx, memória do pod por serviço	alertas por serviço, como na Fase 3

Os alertas passaram a enviar e-mail por meio de um workflow do New Relic, que faltava na Fase 3. Cada alerta aponta para um runbook no repositório do OS Service.

11. Segurança

- **Autenticação:** segundo fator por e-mail, mesma resposta para CPF cadastrado ou não, código guardado como hash, limite de tentativas.
- **Autorização:** JWT validado no gateway e em cada serviço; o cliente só acessa os próprios dados (recurso de outro cliente responde 404).
- **Integrações:** token de decisão do orçamento guardado como hash; notificação do Mercado Pago com assinatura validada; endpoint interno protegido por chave e sem rota pública.
- **Dados:** credenciais de banco por serviço; TLS obrigatório no RDS e no DocumentDB; filas com criptografia; consulta pública de status sem dados pessoais.
- **Execução:** usuário não-root, sistema de arquivos somente leitura, sem escalonamento de privilégio; roles de pod e de pipeline com menor privilégio.

12. Custos

O ambiente continua sendo criado para cada sessão de trabalho e destruído ao final. Em relação à Fase 3, a Fase 4 acrescenta uma instância RDS db.t4g.micro para o Billing Service e uma instância DocumentDB db.t4g.medium para o Execution Service; SQS, DynamoDB e SES têm custo desprezível no volume do projeto.

Item	US$/hora (ligado)
Base da Fase 3 (EKS, 2 nós t3.medium, NAT instance, NLB, RDS do OS)	≈ 0,24
RDS do Billing (db.t4g.micro + 20 GB gp3)	≈ 0,02
DocumentDB do Execution (db.t4g.medium + armazenamento e I/O)	≈ 0,08
SQS, DynamoDB, SES, Lambda	≈ 0
Total	≈ 0,34

Uma sessão de ensaio ou gravação de quatro horas custa cerca de US$ 1,40. O ambiente de produção permanece configurado nos pipelines e não foi provisionado, pela mesma razão de custo.

13. Decisões Arquiteturais e Débitos Técnicos

As decisões da fase foram registradas em ADRs, numeradas em sequência às da Fase 3. A ADR-022 supera a ADR-001 quanto à localização dos manifestos e do banco, e a ADR-019 complementa a ADR-010, cuja rejeição de NoSQL valia para o domínio único da fase anterior.

ADR	Decisão
016	Decomposição em três microsserviços por subdomínio (Strangler Fig) e composição de API
017	Saga orquestrada pelo OS Service
018	Amazon SQS como mensageria
019	Banco por serviço: PostgreSQL, MongoDB (DocumentDB) e DynamoDB
020	Transactional Outbox e consumidor idempotente
021	Mercado Pago Checkout Pro com webhook assinado e reconciliação
022	Infraestrutura por serviço, Pod Identity e roteamento na borda
023	Segundo fator na autenticação e desacoplamento do banco do OS

Débito	Situação
DT-06 Autenticação só por CPF	encerrado pela ADR-023
DT-07 Tabela refresh_tokens órfã	encerrado (tabela removida)
DT-12 Manifestos aplicados pelo Terraform	encerrado pela ADR-022
DT-10 Produção nunca provisionada	mantido por custo
DT-02 Segredo JWT simétrico compartilhado	mantido; agora com mais consumidores
Novo — código transversal replicado em três repositórios	aberto
Novo — isolamento de rede entre pods e bancos só por credencial	aberto
Novo — DocumentDB e RDS em instância única	aberto
Novo — roles de pipeline com edição no mesmo namespace	aberto
Novo — saga em FALHA exige intervenção manual	aberto, com runbook
Novo — outbox e inbox sem expurgo	aberto

14. Conclusão

A aplicação foi dividida em três serviços com repositório, infraestrutura e banco próprios. O monólito evoluiu para o OS Service, sem perder o histórico, e os novos serviços de orçamento e pagamento e de execução assumiram as responsabilidades extraídas. A ordem de serviço passou a ser coordenada por uma saga orquestrada, com compensações ordenadas

conforme as contramedidas da disciplina e entrega garantida por outbox e handlers idempotentes.

A persistência passou a combinar PostgreSQL, MongoDB e DynamoDB, cada um escolhido pelo formato dos dados e pelo tipo de consistência necessário. A integração com o Mercado Pago inclui notificação assinada, reconciliação e estorno como compensação.

Os pontos registrados nas avaliações anteriores foram tratados: o Domain Storytelling está na documentação, os contêineres executam sem privilégio de root e a autenticação tem segundo fator. A produção permanece configurada e não provisionada por custo.

15. Referências Bibliográficas

AMAZON WEB SERVICES. Transactions in Amazon DocumentDB. Disponível em: https://docs.aws.amazon.com/documentdb/latest/developerguide/transactions.html. Acesso em: 29 out. 2026.

AMAZON WEB SERVICES. Amazon EKS Pod Identity. Disponível em: https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html. Acesso em: 29 out. 2026.

ASYNCAPI INITIATIVE. AsyncAPI Specification 2.6.0. Disponível em: https://www.asyncapi.com/docs/reference/specification/v2.6.0. Acesso em: 29 out. 2026.

EVANS, E. Domain-Driven Design: atacando as complexidades no coração do software. 3. ed. Rio de Janeiro: Alta Books, 2016.

FOWLER, M. Strangler Fig Application. 2004. Disponível em: https://martinfowler.com/bliki/StranglerFigApplication.html. Acesso em: 29 out. 2026.

HOFER, S.; SCHWENTNER, H. Domain Storytelling: a collaborative, visual, and agile way to build domain-driven software. Boston: Addison-Wesley, 2021.

MERCADO PAGO. Checkout Pro — notificações de pagamento. Disponível em: https://www.mercadopago.com.br/developers/pt/docs/checkout-pro/payment-notifications. Acesso em: 29 out. 2026.

MERCADO PAGO. Checkout Pro — compras de teste. Disponível em: https://www.mercadopago.com.br/developers/pt/docs/checkout-pro/integration-test/test-purchases. Acesso em: 29 out. 2026.

NEW RELIC. Instrument AWS SQS message queues (Java agent). Disponível em: https://docs.newrelic.com/docs/apm/agents/java-agent/instrumentation/instrument-aws-sqs-message-queues/. Acesso em: 29 out. 2026.

RICHARDSON, C. Microservices Patterns: with examples in Java. Shelter Island: Manning, 2018.

RICHARDSON, C. A pattern language for microservices. Disponível em: https://microservices.io/patterns/. Acesso em: 29 out. 2026.

SADALAGE, P. J.; FOWLER, M. NoSQL Distilled: a brief guide to the emerging world of polyglot persistence. Boston: Addison-Wesley, 2012.

FIAP. Material das disciplinas da Fase 4 — Data Engineering; Banco de Dados em Grafos; Estrutura de Microsserviços; Estrutura de Microsserviços Parte II; SAGA Pattern. PosTech Arquitetura de Software, 2026.